ARQUITECTURA DE SOFTWARE

Taller del 2do corte · Docker

Informe de investigación

Fundamentos de Docker · Parte 1

Estudiante

Juan Jose Barreto

Sistema operativo

Windows

Octubre de 2026

Introducción

Hoy casi cualquier aplicación se entrega empaquetada en contenedores. Docker es la herramienta que popularizó esta idea. En este informe explico qué es Docker, de qué se compone, qué puede hacer, sus comandos principales, el problema que resuelve y por qué importa en una arquitectura de software, donde cada servicio (API, base de datos, broker) suele correr en su propio contenedor.

Desarrollo

1. ¿Qué es Docker?

Docker es una plataforma que permite empaquetar una aplicación junto con todo lo que necesita para funcionar (librerías, runtime, configuración) en una unidad llamada **contenedor**, que se ejecuta igual en cualquier máquina que tenga Docker.

Cómo funciona a alto nivel: un contenedor es un proceso aislado dentro del computador. El sistema operativo le muestra solo su propio "mundo": sus archivos, su red y sus procesos. Como todos los contenedores **comparten el kernel del sistema operativo del host**, no necesitan arrancar un sistema operativo completo cada uno.

Contenedor vs. máquina virtual: una máquina virtual incluye un sistema operativo completo encima de un hipervisor, por lo que pesa gigabytes y tarda en arrancar. Un contenedor solo lleva la aplicación y sus dependencias, pesa megabytes y arranca en segundos. Por eso es más ligero.

2. Componentes

- **CLI (`docker`):** el cliente por el que escribo comandos.
- **Daemon (`dockerd`):** el motor que corre en segundo plano y hace el trabajo real (construir imágenes, crear y ejecutar contenedores).
- **Imagen:** plantilla de solo lectura con todo lo necesario para ejecutar una aplicación.
- **Contenedor:** una imagen en ejecución.
- **Dockerfile:** archivo de texto con las instrucciones (la "receta") para construir una imagen.
- **Registro (Docker Hub):** almacén donde se descargan y publican imágenes.

Cadena principal:

```
Dockerfile --(docker build)--> Imagen --(docker run)--> Contenedor
                                 ^  |
                     docker pull|  |docker push
                                |  v
                          Registro (Docker Hub)
```

El CLI le envía órdenes al daemon. El daemon construye la imagen a partir del Dockerfile, y si la imagen base no existe localmente, la descarga del registro. Luego crea contenedores a partir de esa imagen.

3. Funcionalidades principales

1. **Portabilidad:** la misma imagen corre igual en mi portátil, en el computador de un compañero o en un servidor en la nube. *Ejemplo:* la imagen de `biblioteca-api` funciona igual en mi computador con Windows que en el de un compañero o en un servidor, sin cambios.

2. **Aislamiento:** cada aplicación vive en su contenedor sin interferir con otras. *Ejemplo:* puedo tener MySQL 5.7 y MySQL 8 corriendo a la vez sin conflicto.
3. **Versionado de imágenes:** las etiquetas permiten elegir versiones exactas. *Ejemplo:* `mysql:8` o `node:18`.
4. **Reproducibilidad del entorno:** todo el equipo usa exactamente el mismo entorno, definido en archivos (Dockerfile, compose). *Ejemplo:* un compañero nuevo clona el repo y levanta todo con un comando.
5. **Escalabilidad:** se pueden arrancar varias instancias del mismo servicio. *Ejemplo:* tres contenedores de la API detrás de un balanceador.
6. **Base para CI/CD:** se construye la imagen automáticamente y se despliega la misma imagen que se probó.

4. Comandos principales

a) Tabla de comandos

Comando	¿Qué hace?	Ejemplo
`docker pull`	Descarga una imagen del registro	`docker pull mysql:8`
`docker run`	Crea y ejecuta un contenedor desde una imagen	`docker run -d --name web -p 8080:80 nginx`
`docker ps`	Lista contenedores en ejecución	`docker ps`
`docker ps -a`	Lista todos los contenedores, incluso detenidos	`docker ps -a`
`docker logs`	Muestra los logs de un contenedor	`docker logs -f web`
`docker exec`	Ejecuta un comando dentro de un contenedor en ejecución	`docker exec -it web sh`
`docker stop` / `start`	Detiene / vuelve a arrancar un contenedor	`docker stop web`
`docker rm`	Elimina un contenedor	`docker rm web`
`docker images`	Lista las imágenes locales	`docker images`
`docker rmi`	Elimina una imagen	`docker rmi nginx`
`docker build`	Construye una imagen desde un Dockerfile	`docker build -t mi-app .`
`docker compose up`	Levanta todos los servicios definidos en `docker-compose.yml`	`docker compose up --build`

b) Tres comandos explicados en detalle

`docker run`: parámetros comunes: - `-d`: ejecuta en segundo plano. - `-p 8080:80`: mapea el puerto 8080 de mi máquina al 80 del contenedor. - `-e VAR=valor`: define una variable de entorno. - `-v datos:/ruta`: monta un volumen para persistir datos. - `--name`: le da un nombre al contenedor.

Cuándo lo uso: para probar rápido un servicio, por ejemplo una base de datos MySQL con `docker run -d --name db -e MYSQL_ROOT_PASSWORD=secreto -p 3306:3306 mysql:8`.

`docker exec`: ejecuta un comando dentro de un contenedor que ya está corriendo. Con `-it` abre una terminal interactiva. *Cuándo lo uso:* para depurar, por ejemplo entrar a la API con `docker exec -it api sh` y revisar archivos o variables.

`docker compose up`: lee `docker-compose.yml` y levanta todos los servicios, con su red y sus volúmenes. `--build` fuerza a reconstruir las imágenes y `-d` lo deja en segundo plano. *Cuándo lo uso:* cuando mi aplicación tiene varios contenedores (API + base de datos), como en `biblioteca-api`.

5. ¿Qué problema resuelve?

Es el clásico **"en mi máquina funciona"**. Una aplicación puede funcionar en el computador del desarrollador y fallar en otro por: versiones distintas de dependencias o del lenguaje, diferencias de sistema operativo, configuraciones distintas y pasos de instalación manual que alguien olvidó.

Antes: para correr una API en Go con MySQL, un compañero nuevo debía instalar Go (versión correcta), instalar y configurar MySQL, crear la base de datos, ejecutar scripts SQL y crear variables de entorno. Cualquier diferencia de versión podía dar errores.

Después: instala Docker, clona el repo, copia el `.env` y ejecuta `docker compose up --build`. El entorno es el mismo para todos porque está descrito en archivos.

6. Importancia en la Arquitectura de Software

- **Un servicio por contenedor:** API, base de datos y broker corren separados, con sus propias dependencias. Esto da aislamiento, permite actualizar uno sin tocar otros y reduce el impacto de fallos.
- **Despliegue consistente:** la misma imagen pasa de desarrollo a pruebas y a producción, así que se elimina la diferencia de entornos.
- **Escalabilidad y nube:** escalar es arrancar más contenedores del servicio que lo necesite. Además, casi todos los proveedores de nube ejecutan contenedores, lo que facilita migrar.
- **Relación con estilos arquitectónicos:** en **microservicios**, cada microservicio es naturalmente un contenedor. En **SOA** facilita desplegar y aislar servicios. En arquitecturas **orientadas a eventos**, el broker de mensajes (por ejemplo RabbitMQ o Kafka) se levanta como un contenedor más, junto a los productores y consumidores.

Conclusión

Usaría Docker en el proyecto `biblioteca-api` del curso (API en Go con MySQL), y en general en cualquier proyecto que tenga una API y una base de datos. Sin Docker, cada persona del equipo tendría que instalar y configurar Go y MySQL por su cuenta, con el riesgo de que las versiones difieran. Con Docker, el entorno queda descrito en archivos y se levanta con un solo comando igual en todas las máquinas. Lo que más me llamó la atención es que un contenedor no necesita un sistema operativo completo, y por eso es tan rápido y ligero comparado con una máquina virtual.

Referencias

1. Docker Inc. *Docker overview*. https://docs.docker.com/get-started/docker-overview/
2. Docker Inc. *Docker CLI reference*. https://docs.docker.com/reference/cli/docker/
3. Docker Inc. *Docker Compose overview*. https://docs.docker.com/compose/
4. Docker Hub. https://hub.docker.com
5. Carreño, repositorio `biblioteca-api` (README). https://github.com/carrenoutb/biblioteca-api